

ISCTE – Instituto Universitário de Lisboa

Escola de Tecnologias Digitais Aplicadas

Análise de uma Rede Social de Programadores GitHub com Teoria de Grafos e Redes Complexas

Projeto da unidade curricular
Introdução a Redes Neurais

Docente: José Barros

Realizado por:

Rafaela Ferreira Nº 110826 – TDIA

Maria Costa Nº 130760 - TDIA

Rui Alves Nº 130755 – TDIA (desistiu)

Rafael Hacamo Nº 130736 – TDIA (desistiu)

Ano letivo 2025/2026

Sintra, 2026

Índice

1. Definição do problema	3
2. Revisão bibliográfica inicial.....	3
3. Identificação e tratamento dos dados.....	3
4. Configuração do sistema em rede.....	4
5. Questão de investigação.....	5
6. Metodologia geral.....	5
7. Tarefas de metodologia e métricas.....	5

1. Definição do problema

Este projeto estuda como se organiza e como funciona uma rede de influência entre programadores na plataforma GitHub. Pretende-se perceber se alguns programadores ocupam posições estruturalmente mais importantes na rede, se existem *hubs* e se há diferenças na estrutura de ligações entre programadores de desenvolvimento Web e de Machine Learning. Em termos simples, o problema pode ser formulado da seguinte forma: dado um conjunto de utilizadores ligados por relações de seguimento mútuo, como podemos representar esta rede como um grafo, identificar posições de maior importância e relacionar essas posições com o tipo de programador (Web ou ML).

Para isso, o projeto vai usar conceitos básicos de teoria de grafos e redes complexas, como nós, arestas e centralidade. A análise será feita sobre uma rede real de programadores GitHub, onde cada nó representa um utilizador e as ligações representam relações sociais na plataforma.

2. Revisão bibliográfica inicial

A teoria de grafos e redes complexas mostra que muitas redes sociais apresentam distribuições de grau assimétricas, com poucos *hubs* e muitos nós pouco ligados. Segundo Newman (2010), estas redes tendem a ter caminhos curtos e formação de triângulos (*small-world*), onde nós centrais têm maior capacidade de difundir informação ou ligar grupos. Medidas de centralidade, como grau ou intermediação, permitem identificar agentes mais influentes (Newman, 2010).

No contexto das redes sociais digitais, vários autores discutem como a posição de um indivíduo na rede se relaciona com o seu impacto sobre os outros, mostrando que estruturas de ligação moldam identidades, opiniões e comportamentos (Pompei, 2018). Em plataformas como o GitHub, seguir outro programador pode ser visto como um tipo de influência profissional, que mostra um reconhecimento de competência ou interesse em aprender com o seu trabalho.

Estudos sobre redes, como o de Rozemberczki et al. (2019), sugerem que existe uma relação profunda entre a estrutura das arestas e os *labels* dos nós. O conceito de homofilia, tendência de nós semelhantes se conectarem preferencialmente, é uma base essencial para este estudo. Ao analisar este dataset, pretendemos verificar, se os padrões de ligação são suficientemente distintos para diferenciar os grupos de Web Development e Machine Learning, focando apenas na morfologia do grafo (Rozemberczki et al., 2019).

3. Identificação e tratamento dos dados

Os dados usados neste trabalho provêm do conjunto “GitHub Social Network (musae-github)”, disponibilizado pelo Stanford Network Analysis Project (SNAP). Trata-se de uma grande rede social de programadores GitHub recolhida através da API pública em junho de 2019. Neste dataset, cada nó/vértice corresponde a um programador que colocou

“star” em pelo menos 10 repositórios. As arestas representam relações de “seguidores”, onde dois utilizadores se seguem mutuamente.

Neste projeto serão utilizados dois ficheiros principais para a análise prática. O ficheiro “musae_git_edges.csv” contém pares de IDs de utilizadores que se seguem mutuamente e será usado para montar a estrutura do grafo. O ficheiro “musae_git_target.csv” associa a cada ID o respetivo nome de utilizador e uma classe binária (desenvolvedor Web ou de Machine Learning). Estes rótulos serão usados como base para comparar estruturalmente os dois grupos de nós. Embora o dataset original também inclua o ficheiro “musae_git_features.json” com características técnicas, este trabalho focará a sua análise na topologia e conectividade da rede.

Para tratar os dados: verificámos se existiam valores nulos e se o tipo de dados estava correto, organizámos por IDs e confirmámos que não existiam arestas repetidas ou auto-ligações em “musae_git_edges.csv” e garantimos que todos os IDs das arestas em “musae_git_edges.csv” também existiam em “musae_git_target.csv”. Esta limpeza serve para garantir que o grafo utilizado é simples, consistente e adequado ao âmbito do projeto.

4. Configuração do sistema em rede

Do ponto de vista técnico, o sistema será representado por um grafo não dirigido $G = (V, E)$. O conjunto de vértices V é formado pelos IDs dos programadores presentes no dataset, onde cada vértice corresponde a um utilizador do GitHub. Esse utilizador tem associados dois atributos principais, provenientes do ficheiro “musae_git_target”, o seu nome de utilizador e um rótulo binário, que indica se o perfil corresponde a um programador de desenvolvimento Web ou de Machine Learning. O conjunto de arestas E corresponde às ligações de seguimento entre utilizadores. É construído a partir do ficheiro “musae_git_edges.csv”, que contém pares não orientado $\{u, v\}$, indicando que relação entre o utilizador u e o utilizador v é recíproca.

Neste trabalho, o grafo é não ponderado, ou seja, todas as ligações têm o mesmo peso, tal como explicitado no documento README.txt do dataset. As arestas são não orientadas e simples, o que implica que não serão consideradas múltiplas ligações entre o mesmo par de nós nem auto-ligações (loops).

A configuração do sistema em rede consiste assim em: construir a estrutura do grafo a partir do ficheiro de “musae_git_edges” e mapear para cada nó o respetivo rótulo social (Web ou ML) a partir do ficheiro “musae_git_target”. Esta configuração define de forma clara os elementos de base, vértices, arestas e rótulos, que serão usados nas fases seguintes do trabalho para analisar a topologia da rede e discutir questões de influência e posição relativa dos programadores no ecossistema do GitHub (Newman, 2010; Pralat et al., 2022). Ao ver a estrutura do grafo concluímos que este é totalmente conexo, tem 37700 vértices (utilizadores), 289003 arestas (ligações) e que, aproximadamente, 74% desses utilizadores pertencem a Web Development e 26% a Machine Learning.

5. Questão de investigação

“Em que medida a estrutura da rede GitHub revela padrões de homofilia e especialização entre developers Web e ML e como é que métricas de redes complexas podem ajudar a explicar e a melhorar a classificação binária destes developers?”

6. Metodologia geral

O objetivo geral do trabalho é caracterizar a estrutura da rede social GitHub e avaliar de que forma as métricas de redes complexas permitem identificar padrões de homofilia e especialização entre developers, Web e ML, e diferenciar a classificação binária dos nós entre estas duas categorias.

Para concretizar este objetivo geral, organizámos a metodologia em objetivos específicos que fomos cumprindo:

1. Descrevemos a rede em termos de dimensão, conectividade e distribuição dos rótulos Web/ML, garantindo a integridade estrutural do grafo utilizado na análise;
2. Caracterizamos a conectividade local dos nós através do grau e analisar se o tipo de developer (Web vs ML) influencia os padrões de ligação na rede;
3. Identificar nós estruturalmente centrais e “pontes” na rede utilizando métricas de centralidade (betweenness, closeness e PageRank), explorando diferenças entre developers Web e ML;
4. Medir a assortatividade de grau e de atributo (Web/ML), bem como o coeficiente de clustering, estudando se a rede apresenta correlações estruturais e padrões de homofilia entre developers do mesmo tipo;
5. Detetar comunidades na rede e analisar a sua composição (Web dominante, ML dominante, mista), relacionando-a com os padrões de homofilia e especialização.

7. Tarefas da metodologia e métricas

Para atingir estes objetivos depois de já ter limpo, organizado, criado o nosso grafo e sincronizado o target, foram-se aplicando as seguintes métricas e algoritmos, implementadas em Python num Jupyter Notebook, recorrendo às bibliotecas Pandas e NetworkX:

Estudo de Componentes: Para averiguar como poderíamos abordar o nosso estudo começámos por calcular o número de componentes conexas, com a função `number_connected_components()` do NetworkX, que apenas era uma.

Grau dos vértices: Para caracterizar a conectividade local da rede, calculou-se o grau de cada nó e analisaram-se estatísticas descritivas para perceber o número médio de ligações por utilizador e a variabilidade desses valores. Em seguida, observou-se a distribuição dos graus no grafo completo e vimos de forma separada como estes valores mudavam por tipo de developer (Web vs ML). Assim conseguimos analisar se o tipo de developer influencia o número de conexões e a forma como estas se distribuem na rede.

Centralidade de grau– Para consolidar o que já tinha sido visto calculámos a centralidade de grau de cada nó para perceber o que aqueles valores significavam em escala e o quão influentes eram os utilizadores mais influentes a nível global. Vimos as métricas a comparar os developers ML e Web e detetámos o top 10 mais central e onde pertenciam.

Centralidade de intermediação (betweenness centrality) – Utilizou-se o algoritmo padrão de betweenness em NetworkX para medir quantas vezes um nó aparece em caminhos entre pares de outros nós, e entender quais são as pontes da rede. Assim, analisam-se que perfis centrais que ligam sub-comunidades, para perceber padrões de ligação e áreas de intersecção entre comunidades. Também calculámos o top 10 com maior central betweenness e pudemos perceber se estes utilizadores “ponte” são os mesmos que utilizadores de maior grau que já tínhamos encontrado.

Centralidade de proximidade (closeness centrality) – Para avaliar quão próximos se encontram os developers relativamente ao resto da rede, calculou-se a centralidade de proximidade para cada nó. Esta métrica foi incluída porque permite distinguir perfis estruturalmente “centrais” daqueles que se encontram mais periféricos, o que pode estar associado a diferentes formas de participação na comunidade. Comparámos novamente entre tipo de developer e vimos os utilizadores mais centrais segundo esta métrica.

PageRank – Aplicou-se ainda o algoritmo PageRank adaptado a grafos não dirigidos, para ter uma noção da importância relacional que não depende apenas do número de ligações, mas também da centralidade dos vizinhos. Com o PageRank podemos testar se developers Web ou ML tendem a concentrar mais “prestígio” na rede de seguidores mútuos, o que pode ter implicações na visibilidade e influência de cada grupo dentro do GitHub. Assim voltámos a ver as estatísticas comparatórias entre utilizadores Web e ML e ver quem são os mais influentes.

Assortatividade de grau e de atributo- Calculámos a assortatividade de grau, para medir se utilizadores com muitas conexões tendem a ligar-se a outros de elevado grau (assortatividade positiva) ou aos de grau baixo (dissortatividade) e vice-versa. A seguir calculámos a assortatividade de atributo, tendo como atributo o rótulo binário Web/ML, para avaliar quantitativamente a homofilia: valores positivos indicam que os utilizadores tendem a ligar-se a outros do mesmo tipo e baixos o contrário.

Clustering: Cálculo do coeficiente de clustering (global e médio por grupo Web e ML) para avaliar o grau de formação de triângulos e subestruturas coesas, verificando se a rede apresenta uma estrutura mais localmente densa em torno de determinados tipos de developers. Isto complementa a análise de homofilia ao mostrar se os laços entre developers do mesmo tipo estão organizados em triângulos e subestruturas coesas.

Deteção de comunidades – Depois detetámos comunidades, primeiro através do algoritmo Girvan-Newman, mas como este é muito pesado e fragmentador e não estava a conseguir detetar comunidades utilizámos o algoritmo Louvain. Esta etapa é crucial para estudar a homofilia, pois permite verificar se as comunidades estruturais são maioritariamente compostas por nós da mesma classe (Web ou ML) e até que ponto os limites entre comunidades acompanham a fronteira entre tipos de especialização.

Referências

- SNAP: Stanford Network Analysis Project. (2018, December 31). *GitHub social network* (musaegithub) [Data set] [<https://snap.stanford.edu/data/github-social.html>]
- Newman, M. E. J. (2010). *Networks: An introduction*. Oxford University Press.[cs.cornell]
- Pompei, M. (2018). Redes sociais: influência, identidade e diferença na sociedade contemporânea. *Em Sociedade*, 3(1), 45–60.[periodicos.pucminas]
- Rozemberczki, B., Allen, C., & Sarkar, R. (2019). *Multi-scale Attributed Node Embedding*. [[ArXiv:1909.13021](https://arxiv.org/abs/1909.13021).]